



南开大学
Nankai University

《软件测试与维护》实验报告

(2025~2026 学年第二学期)

实验名称：微服务系统 SockShop 部署实验

学 院：软件学院

姓 名：李宗杭

学 号：2311451

指导老师：张圣林

2026 年 5 月 13 日

目录

1 实验背景	1
2 实验内容	1
3 实验步骤	1
3.1 安装 WSL2 与 Docker Desktop	1
3.2 安装 Minikube 并启动集群	2
3.3 部署 SockShop 微服务应用	4
3.4 访问前端界面	5
4 实验结果	6
5 实验总结与心得体会	6
5.1 遇到的问题及解决方法	6
5.2 心得感悟	6

实验 1：基于 Docker 和 Minikube 的微服务系统 SockShop 部署实验

1 实验背景

随着云计算和微服务架构的普及，容器化技术（如 Docker）和容器编排平台（如 Kubernetes）已成为现代应用部署的核心。本次实验旨在通过本地轻量级 Kubernetes 环境（Minikube），部署一个经典的微服务示例应用——SockShop，从而理解 Docker、Kubernetes、Pod、Service、命名空间等核心概念，并掌握基于容器的微服务系统部署流程。

2 实验内容

本次实验在 Windows 操作系统上完成以下任务：

1. 安装并配置 Docker Desktop（使用 WSL2 后端）；
2. 安装 Minikube 并创建单节点 Kubernetes 集群；
3. 从 GitHub 克隆 SockShop 项目，并通过 kubectl 部署到 Minikube 集群；
4. 验证所有微服务 Pod 的状态，并通过浏览器访问前端界面；
5. 记录部署过程中遇到的问题及解决方法。

3 实验步骤

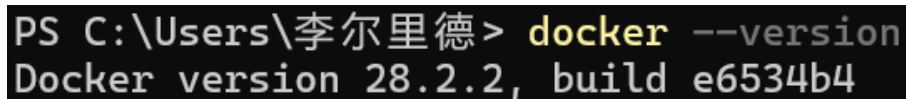
3.1 安装 WSL2 与 Docker Desktop

1. 首先安装 WSL2 并设置为默认版本。在 PowerShell 中以管理员身份运行：

```
1 wsl --set-default-version 2
```

2. 下载并安装 Docker Desktop for Windows。
3. 启动 Docker Desktop, 登录 Docker Hub 账户。
4. 验证 Docker 安装:

```
1 docker --version
```

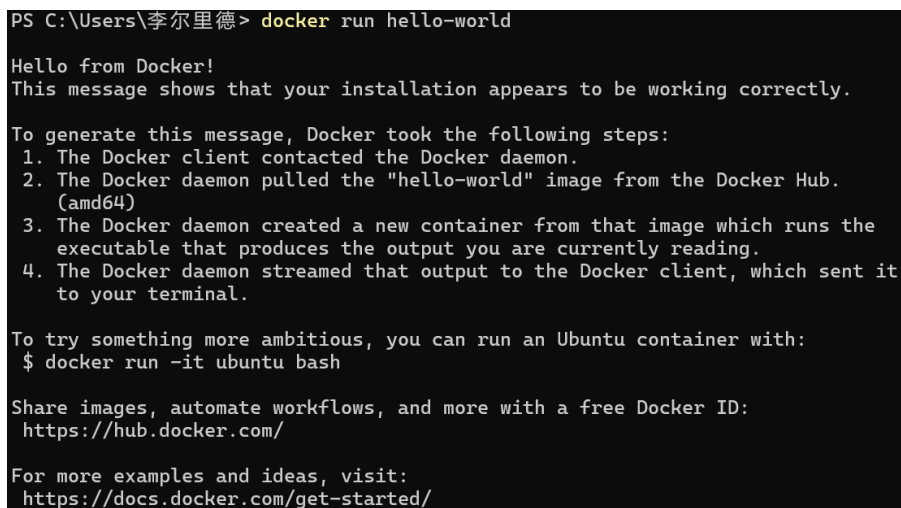


```
PS C:\Users\李尔里德> docker --version
Docker version 28.2.2, build e6534b4
```

图 1: Docker 版本验证命令输出截图

5. 运行 hello-world 容器测试:

```
1 docker run hello-world
```



```
PS C:\Users\李尔里德> docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

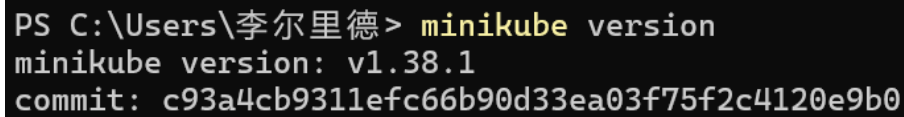
For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

图 2: 运行 hello-world 容器的输出信息

3.2 安装 Minikube 并启动集群

1. 下载 Minikube Windows 版本, 并添加到系统 PATH。
2. 验证 Minikube 安装:

```
1 minikube version
```



```
PS C:\Users\李尔里德> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
```

图 3: Minikube 版本信息截图

3. 启动 Minikube 集群（指定 Docker 驱动，分配 4 核 6G 内存，使用阿里云镜像加速）：

```
1 minikube start
2     --driver=docker
3     --cpus=4
4     --memory=6g
5     --base-image='registry.cn-hangzhou.aliyuncs.com/google_containers/kicbase:v0.0.44'
6
```



```
PS C:\Users\李尔里德> minikube start --driver=docker --cpus=4 --memory=6g --base-image='registry.cn-hangzhou.aliyuncs.com/google_containers/kicbase:v0.0.44'
* Microsoft Windows 11 Home China 25H2 上的 minikube v1.38.1
- MINIKUBE_HOME=D:\Kubernetes\data
* 根据现有的配置文件使用 docker 驱动程序
* 在集群中 "minikube" 启动节点 "minikube" primary control-plane
* 正在拉取基础镜像 v0.0.50 ...
* 正在为 "minikube" 重启现有的 docker container ...-
! 此镜像不适用于当前的 minikube 版本。要解决此问题，您可以删除并重新创建您的 minikube 集群，使用最新的镜像。预期的 minikube 版本: v1.33.1 -> 实际的 minikube 版本: v1.38.1
* 正在 Docker 26.1.1 中准备 Kubernetes v1.35.1/-
* 正在验证 Kubernetes 组件...
- 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
* 启用插件: storage-provisioner, default-storageclass

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe 的版本为 1.32.2, 可能与 Kubernetes 1.35.1 不兼容。
- 想要使用 kubectl v1.35.1 吗? 尝试使用 'minikube kubectl -- get pods -A' 命令
* 完成! kubectl 现已配置，默认使用"minikube"集群和"default"命名空间
```

图 4: Minikube 集群启动截图

4. 等待集群启动完成，检查状态：

```
1 minikube status
```

```
PS C:\Users\李尔里德> minikube status
minikube
type: Control Plane
host: Running
kublet: Running
apiserver: Running
kubeconfig: Configured
```

图 5: Minikube 集群状态

3.3 部署 SockShop 微服务应用

1. 克隆 SockShop 项目仓库:

```
1 git clone https://github.com/microservices-demo/microservices-demo
2 cd microservices-demo
```

2. 创建命名空间并部署所有资源:

```
1 kubectl create -f ...
   deploy/kubernetes/manifests/00-sock-shop-ns.yaml -f ...
   deploy/kubernetes/manifests/
```

3. 查看所有 Service:

```
1 kubectl get svc -n sock-shop
```

```
PS C:\Users\李尔里德> kubectl get svc -n sock-shop
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
carts	ClusterIP	10.107.14.255	<none>	80/TCP	43h
carts-db	ClusterIP	10.101.150.123	<none>	27017/TCP	43h
catalogue	ClusterIP	10.104.176.60	<none>	80/TCP	43h
catalogue-db	ClusterIP	10.104.110.36	<none>	3306/TCP	43h
front-end	NodePort	10.100.187.125	<none>	80:30001/TCP	43h
orders	ClusterIP	10.108.174.176	<none>	80/TCP	43h
orders-db	ClusterIP	10.96.9.182	<none>	27017/TCP	43h
payment	ClusterIP	10.98.116.115	<none>	80/TCP	43h
queue-master	ClusterIP	10.104.180.174	<none>	80/TCP	43h
rabbitmq	ClusterIP	10.111.104.219	<none>	5672/TCP, 9090/TCP	43h
session-db	ClusterIP	10.101.52.196	<none>	6379/TCP	43h
shipping	ClusterIP	10.101.52.19	<none>	80/TCP	43h
user	ClusterIP	10.110.118.46	<none>	80/TCP	43h
user-db	ClusterIP	10.96.92.129	<none>	27017/TCP	43h

图 6: SockShop 所有微服务及其类型、端口信息

4. 查看 Pod 状态，等待所有 Pod 进入 Running 状态

```
1 kubectl get pods -n sock-shop -w
```

```
PS C:\Users\李尔里德> kubectl get pods -n sock-shop -w
```

NAME	READY	STATUS	RESTARTS	AGE
carts-b6c5c87f9-4nnzk	1/1	Running	3 (41h ago)	43h
carts-db-6bd47f898c-zfkgv	1/1	Running	2 (41h ago)	43h
catalogue-744d46f784-b9r77	1/1	Running	3 (41h ago)	43h
catalogue-db-6d4c747cc6-r7n9d	1/1	Running	3 (41h ago)	43h
front-end-87ff7b849-mnxj9	1/1	Running	3 (41h ago)	43h
orders-6cfcf6794c-dvztx	1/1	Running	1 (41h ago)	41h
orders-db-57b97f49c-qgbg4	1/1	Running	2 (41h ago)	43h
payment-7588d7df-vh8r9	1/1	Running	3 (41h ago)	43h
queue-master-f75b7664f-tbgjz	1/1	Running	3 (41h ago)	43h
rabbitmq-fcb976dd-4qbj2	2/2	Running	5 (41h ago)	43h
session-db-5cc78648ff-ghjq6	1/1	Running	3 (41h ago)	43h
shipping-6ddcd77687-tpxwx	1/1	Running	3 (41h ago)	43h
user-9c64594b7-kld9v	1/1	Running	3 (41h ago)	43h
user-db-5b8fd7455-sdmfv	1/1	Running	3 (41h ago)	43h

图 7: 所有微服务 Pod 均处于 Running 状态

3.4 访问前端界面

1. 使用 Minikube 暴露 front-end 服务并自动打开浏览器:

```
1 minikube service front-end -n sock-shop
```

2. 浏览器中应显示 SockShop 的 Web 商店界面。

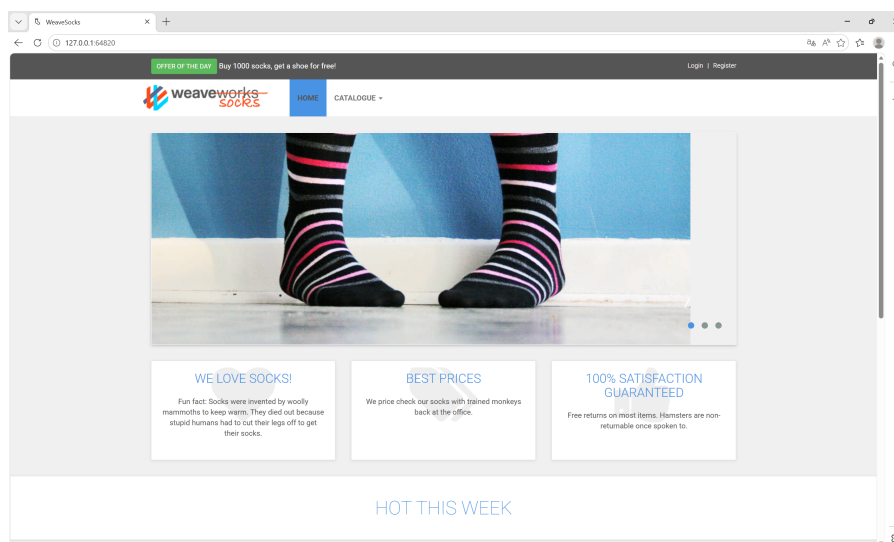


图 8: 浏览器访问 SockShop 前端页面

4 实验结果

成功在 Windows 上通过 Docker Desktop 和 Minikube 部署了 SockShop 微服务系统。所有微服务 Pod 均正常运行，前端界面可通过 NodePort 地址在浏览器中访问，页面显示各类商品信息，交互功能正常。

5 实验总结与心得体会

5.1 遇到的问题及解决方法

1. Minikube start 失败，无法拉取 kicbase 镜像：使用阿里云镜像指定 base-image 参数。
2. Pod 长时间处于 ContainerCreating 或 ImagePullBackOff 状态：开启 vpn 的 tun 模式。

5.2 心得感悟

通过本次实验，深入理解了容器与编排工具的工作流程。Docker 提供了轻量级的运行环境，而 Kubernetes（即便是单节点的 Minikube）则展示了如何管理多个容器的生命周期、服务发现、负载均衡等能力。SockShop 作为经典的微服务示例，清晰展示了每个独立服务的作用以及它们之间的通信方式。此外，实验过程也让我认识到网络环境对部署的影响，学会使用国内镜像是提高成功率的有效手段。